Università degli Studi di Padova

School of Engineering Department of Information Engineering

Master Degree in Computer Engineering

Exact and heuristic approaches to the Travelling Salesman Problem (TSP)

Operations Research 2

Supervisor:
Matteo Fischetti

Candidates:
Fantin Luca (2119287)
Zanella Matteo (2122187)

Accademic Year 2023/2024

Contents

1 Introduction . . . **1**
1.1 Problem formulation . . . 1

2 Project Setup . . . **3**
2.1 Generating the instances . . . 3
2.2 Installing CPLEX . . . 3
2.3 Reading the solution plots . . . 3
2.4 Performance profiler . . . 4

3 Heuristics . . . **5**
3.1 Nearest Neighbor (Greedy) . . . 5
3.1.1 Pseudocode . . . 6
3.1.2 Results analysis . . . 7
3.2 2-opt . . . 7
3.2.1 Pseudocode . . . 8
3.2.2 Swap policy . . . 9
3.2.3 Results analysis . . . 9
3.3 Comparison Greedy / 2-opt . . . 10

4 Metaheuristic . . . **12**
4.1 Tabu Search . . . 12
4.1.1 Storing a Tabu move . . . 12
4.1.2 The tabu list . . . 13
4.1.3 Pseudocode . . . 14
4.1.4 Results analysis . . . 14
4.2 Variable Neighborhood Search (VNS) . . . 14
4.2.1 Results analysis . . . 15
4.3 Comparison Tabu / VNS . . . 15

5 Exact methods . . . **16**
5.1 CPLEX . . . 16
5.2 Benders loop . . . 16
5.2.1 Patching . . . 16
5.3 Branch & Cut . . . 16

5.3.1 Warm start 16
5.3.2 Fractionary solutions 16
5.3.3 Patching 16

6 Matheuristics 17
6.1 Diving 17
6.2 Local Branching 17

7 Bibliography 18

Chapter 1

Introduction

The Travelling Salesman Problem (TSP), is one of the most famous and studied optimization problems in the Operations Research field.
Although its first mathematical formulation was proposed in the 19th century by the mathematicians William Rowan Hamilton and Thomas Kirkman, it received scientific attention from the 1950s onwards.
In 1972, Richard M. Karp proved the NP-hard nature of the TSP; this meant that the computation time for any solving algorithm can grow exponentially with the input size. Despite this, many different approaches have been developed over the years, yielding both exact and approximate solutions.

In this paper, several algorithms are explained, developed and tested against each other, both exact and approximate.

1.1 Problem formulation

Consider an undirected graph $G = (V, E)$, where V is a set of $|V| = N$ nodes (or vertices) and E is a set of $|E| = M$ edges.
Define a Hamiltonian cycle of G, $G^* = (V, E^*)$, as a graph whose edges form a cycle going through each node $v \in V$ exactly once.
Let's also define a cost function for the edges $c : E \to \mathbb{R}^+$, $c_e := c(e)\ \forall\ e \in E$.

The target of the TSP is finding an Hamiltonian cycle of G of minimum total cost, obtained by summing the costs of all edges in the cycle.
This problem can be formulated through an Integer Linear Programming (ILP) model.
First, let's define the following decision variables to represent whether or not a certain edge is included in the Hamiltonian cycle:

$$x_e = \begin{cases} 1 & \text{if } e \in E^* \\ 0 & \text{otherwise} \end{cases} \qquad \forall\ e \in E$$

The ILP model is the following:

$$\begin{cases} \min \sum_{e \in E} c_e x_e & (1.1) \\ \sum_{e \in \delta(h)} x_e = 2 \quad \forall\, h \in V & (1.2) \\ \sum_{e \in \delta(S)} x_e \leq |S| - 1 \quad \forall\, S \subset V : v_1 \in S & (1.3) \\ 0 \leq x_e \leq 1 \quad \text{integer} \quad \forall\, e \in E & (1.4) \end{cases}$$

Constraints 1.2 impose that each node has a degree of 2 ($|\delta(v)| = 2, \forall v \in V$). This group of contraints alone isn't enough to guarantee to find a valid Hamiltonian Cycle: the solution found could be composed by lots of isolated cycles.
Constraints 1.3, called Subtour Elimination Constraints (SEC), guarantee that any solution found through this model is made up of only one connected component: every vertex $v \neq v_1$ must be reachable from v_1.
Despite their importance, their number is exponential in N, thus, considering all of them at once is computationally expensive.

Chapter 2

Project Setup

All the algorithms written in this thesis (to solve the TSP) where written using the C language, both due to its speed and CPLEX support for the language.
Plotting and testing are done with Python3, due to it's extensive plotting libraries and the ease in writing it.

2.1 Generating the instances

The instances used to test the algorithms have been randomly generated.
The problem space is a 2D grid: $[0, 10_000] \times [0, 10_000] \subset \mathbb{R}^2$.
Each point is generated from an i.i.d. uniform distribution of the grid defined earlier.

2.2 Installing CPLEX

To view the exact solution of our problem (optimal solution), we opted for (IBM ILOG) CPLEX as a MIP (Mixed Integer Programming) solver.
To get cplex visit IBM's web page and look for the CPLEX Optimization Studio.
TODO: maybe add something else?

2.3 Reading the solution plots

Each solution found is accompanied by a plot showing (graphically) the solution with some info.
Here's a list of the explanation of the info showed in the plots:

1. The name of the algorithm used:

greedy : Nearest Neighborg greedy algorithm

g2opt : applying 2opt to the greedy solution

tabu : Tabu algorithm

vns : VNS algorithm

cplex : CPLEX exact algorithm

2. In the same line of the algorithm there might be written some parameters used:

g2opt : (f) → g2opt with first swap policy

g2opt : (b) → g2opt with best swap policy

tabu : int-int-double → tabu with fixed_tenure - variable_tenure - variability_frequency parameters

cplex : benders loop → using benders loop

cplex : mipst → giving cplex a "warm" start

cplex : ccb → using the candidate callback

cplex : rcb → using the relaxation callback

cplex : n/g-patch → using patching if the solution provided is disconnected

cplex : ccb/rcb-patch → using patching inside the (respective) callbacks

(each parameter will be explained in the respective section).
If the algorithm exceeded the time limit or has been terminated early by the user, an asterisk (*) will be shown at the end of the first line

3. cost: the cost of the Hamiltonian Cycle found

4. double/double: time needed to find that solution / total computing time

2.4 Performance profiler

To compare different algorithms we used the performance profiles, a tool provied by our professor that plots how often an algorithm is within a percentage of the cost from the best solution found
TODO: Explain better how to read the plot

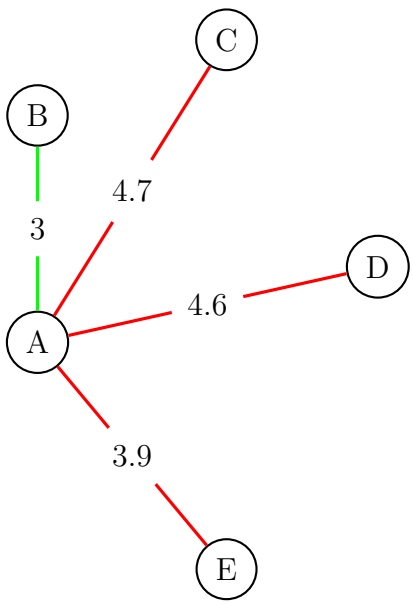
Chapter 3

Heuristics

An heuristic is any approach to problem solving that employs a practical method that's not fully optimized but it's sufficient to reach an immediate short-term goal or approximation.
Given the NP-Hard nature of the Travelling Salesman Problem, finding the optimal solution may require a long time, hence the need to have heuristics method to find solutions that are close to the optimal.

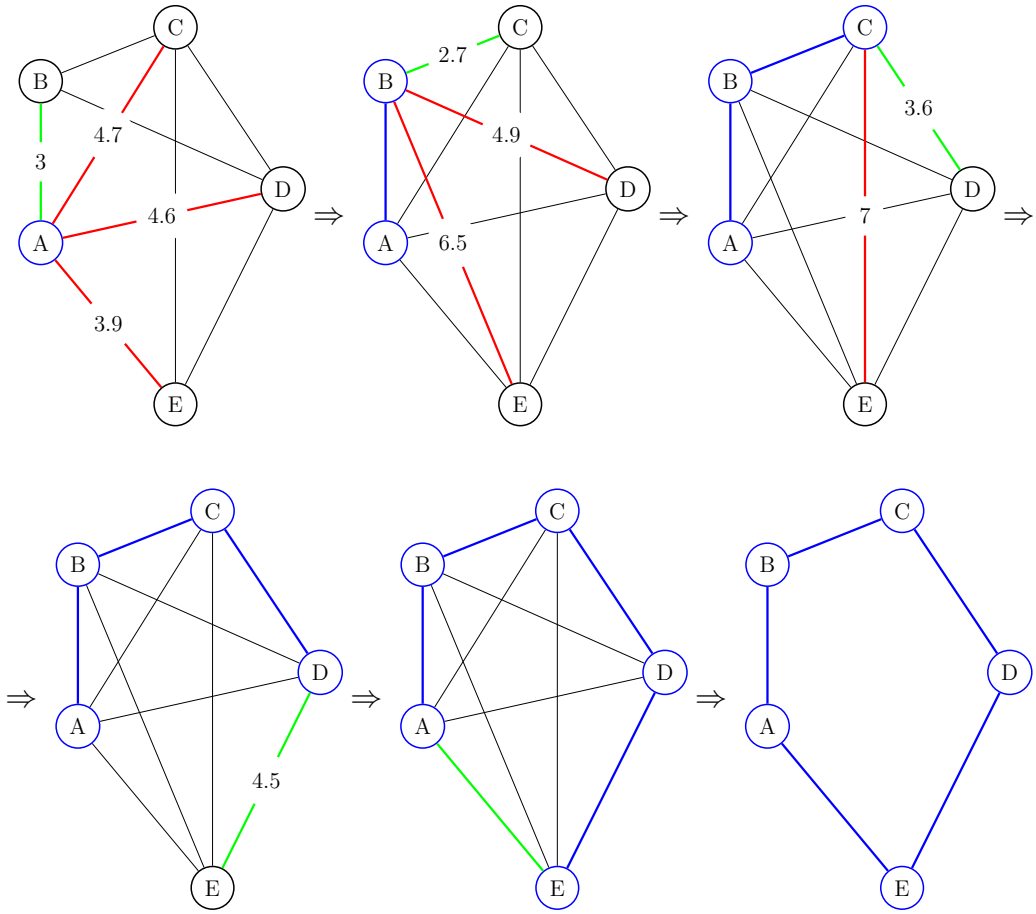
3.1 Nearest Neighbor (Greedy)

A first approach to the TSP, is to consider the edges with the smallest weights first when choosing the next node in the cycle.
This type of logic is called greedy: a greedy algorithm looks for the locally optimal choice at each stage.



In this example, among the edges connected to the node A, the edge (A, B) is the one with the smallest weight, so node B should be A's successor.

By repeating this process for each new node added to the path and connecting the last node (E) to the starting node (A):



3.1.1 Pseudocode

Algorithm 1 Greedy algorithm for the TSP

Input Starting node ($s \in V$), Set of nodes (V)

Output List of $n := |V|$ nodes forming an Hamiltonian cycle, Cost of the cycle

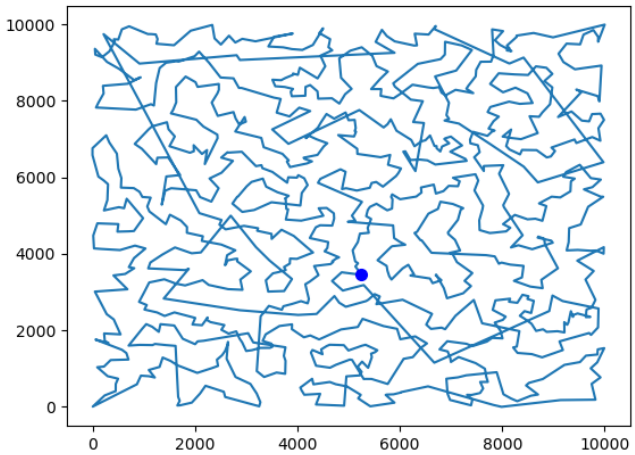
```
cycle ← [s]
cost ← 0
for i = 0 to n − 2 do
    next ← argmin_v {c_{cycle[i],v} | v ∈ V ∧ v ∉ cycle}
    cost ← cost + c_{cycle[i],next}
    cycle[i + 1] ← next
end for
cost ← cost + c_{cycle[n−1],s}

return cycle, cost
```

The solution found using the greedy algorithm is dependent to the starting node: a possible solution to this is to iterate through all possible starting nodes and keeping track of the best solution found so far.

3.1.2 Results analysis

TODO: new plot.

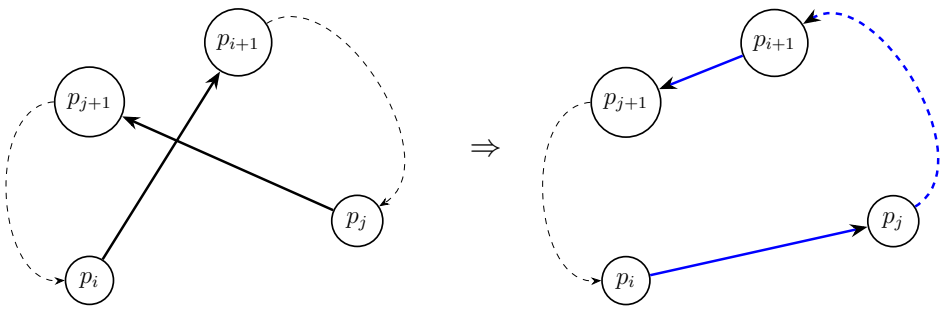


Instance: **A**, Algorithm: **greedy**, Cost: **282030.8675**

The solutions found with the greedy algorithm are a good starting point, but sometimes the algorithm uses edges that cross a long distance, increasing a lot the cost of the solution.
This is caused by the fact that the greedy algorithm optimizes locally, without knowing wether that local choice is good or not in the long term: it might happen that after adding some edges to the cycle, the closest nodes are all already in the cycle, so the edge that will be considered may have a very large weight.
In the next section a tecnique is explored that will fix this problem.

3.2 2-opt

The 2-opt algorithm takes an existing cycle and tries to improve it's cost by changing some of the edges that compose the cycle without braking it.
The main idea behind the 2-opt algorithm is to find two edges that cross each other and fix them by removing the intersection:



$p_i := \text{cycle}[i]$

This process is then repeated until no more edges that can lead to an improvement to the cost of the cycle can be found.
To find a pair of edges that can be changed to improve the cost it's sufficient to find a pair of nodes p_i, p_j, that satisfies the following inequality:

$$c_{p_i,p_{i+1}} + c_{p_j,p_{j+1}} > c_{p_i,p_j} + c_{p_{i+1},p_{j+1}}$$

Note that the cycle list is to be considered as a circular array: situations with indexes out of bounds or $i > j$ won't be treated in this paper since the fix is trivial.

If this inequality holds then swapping the edges (p_i, p_j), (p_{i+1}, p_{j+1}) with (p_i, p_{i+1}), (p_j, p_{j+1}) will lower the cost of the cycle:

$$\begin{aligned}\text{cost(new cycle)} &= \text{cost(old cycle)} - (c_{p_i,p_{i+1}} + c_{p_j,p_{j+1}}) + (c_{p_i,p_j} + c_{p_{i+1},p_{j+1}}) \\ &\leq \text{cost(old cycle)}\end{aligned}$$

After finding the nodes p_i, p_j the edges (p_i, p_j) and (p_{j+1}, p_{j+1}) will take the place of the edges (p_i, p_{i+1}) and (p_j, p_{j+1}), reversing the route connecting $p_{i+1} \to p_j$.
Suppose the cycle is stored as a list of nodes ordered following the order of the nodes in the cycle, then this step can be done simply by reversing the list from the index $i + 1$ to the index j:

$$[..., p_i, \underline{p_{i+1}, ..., p_j}, p_{j+1}, ...] \Rightarrow [..., p_i, \underline{p_j, ...(\text{reversed})..., p_{i+1}}, p_{j+1}, ...]$$

3.2.1 Pseudocode

Algorithm 2 2-opt algorithm for the TSP

Input List (cycle) of $n := |V|$ nodes forming an Hamiltonian cycle, Cost (cost) of the cycle

Output List of n nodes forming an Hamiltonian cycle (with 2-opt swaps applied), Cost of the new cycle

```
while *Exists a swap improving the cost* do
    (i, j) ← find_swap(cycle)
    cost ← cost − (c_{p_i,p_{i+1}} + c_{p_j,p_{j+1}}) + (c_{p_i,p_j} + c_{p_{i+1},p_{j+1}})
    reverse(cycle, i + 1, j)
end while

return cycle, cost
```

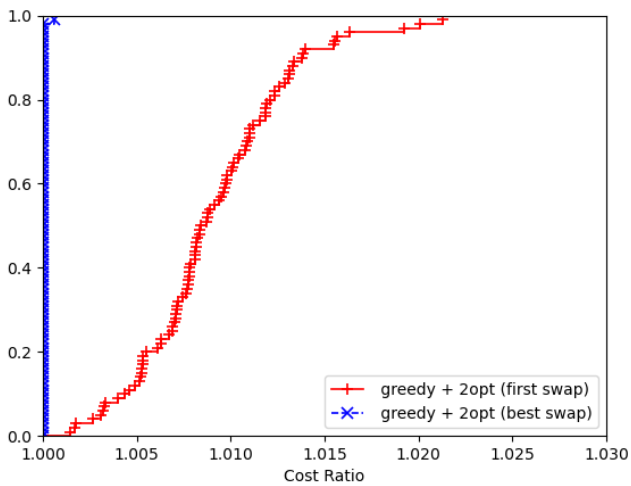
Where the find_swap method finds a swap that improves the cost.

3.2.2 Swap policy

In this paper two different ways of finding a swap have been confronted:

- Returning the first swap found that improves the cost
- Looking among all possible pair of edges and returning the swap that improves the cost the most (the best swap)

Using the performance profiler it's easy to see that the best swap policy finds solutions with an improvement of a 2% factor with respect to the first swap policy:
TODO: new plot



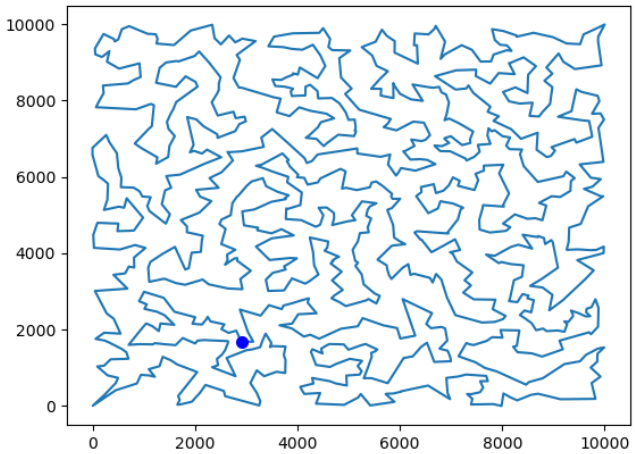
100 instances, Time limit: 120s

This is due to the fact that the first swap policy might get lost in improving the solution by a small amount by finding swaps in a region with an high concentration of nodes while the best swap policy aims to improve the edges with the highest weights.
This leads the best swap policy to fix immediately the worst edges, lowering right away the cost of the solution by a large amount.
It's also worth noticing that the the first swap policy requires less time to improve the solution

3.2.3 Results analysis

TODO: new plot

The 2-opt algorithm is usually used after finding a cycle with the greedy algorithm to see wether it can be improved.
The solution displayed is obtained by running the 2-opt algorithm, after applying the greedy algorithm on each possible starting node, and keeping the best one.

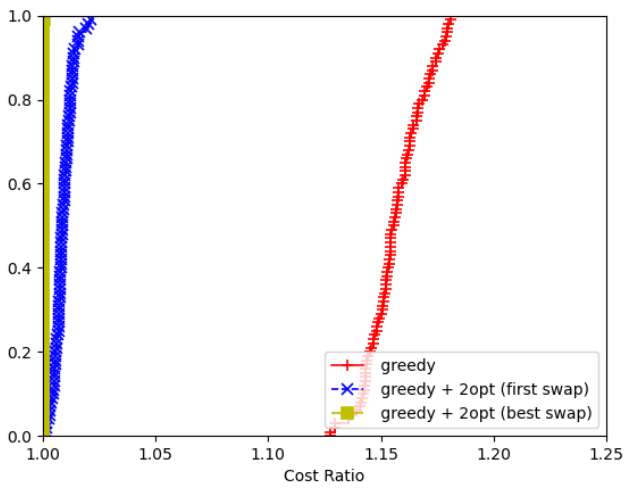


Instance: **A**, Algorithm: **greedy + 2-opt (best swap)**, Cost: **244793.5477**

Confronting this solution with the one showed for the greedy algorithm, it's possible to notice that the starting node used for the greedy algorithm has changed: even though the greedy algorithm found that starting node to be the best one (the one that generates the best cycle), after improving the solution using the 2-opt algorithm, another starting node has been found to be better.
This is a phenomenon quite common in heuristics: when trying to improve two different solutions, improving the worse solution can lead to better results than improving the better one. This concept will be further explored in the metaheuristics chapter.

3.3 Comparison Greedy / 2-opt

TODO: new plot



100 instances, Time Limit: 120s

The 2-opt algorithm manages to consistently improve the cost of the solutions of 15-

20% with respect to the solutions found by the greedy algorithm.

TODO: Add cost comparison with exact algorithms.

Chapter 4

Metaheuristic

A metaheuristic approach is a method in which more than one heuristic is used, with the aim to guide the search processs to efficiently explore the search space.
Metaheuristics will unlock a greater search space with respect to heuristic, by allowing "bad moves" to escape a locally optimal solution.

4.1 Tabu Search

The Tabu search algorithm is based on the idea of allowing the 2-opt algorithm to perform swaps that still are the best ones, but not necessarly swaps that improve the cost of the solution.
This means that after finding a local optima, the 2-opt algorithm will stop, while the tabu search algorithm will keep searching, moving away from that locally optimal solution, hoping to find a new locally optimal solution with a lower cost.
Allowing a bad move, means that at the next iteration the new best move will revert it, since that would be the only swap that lowers the cost.
To prevent this, we need to keep track of those bad moves and prevent them from being reverted, marking them as tabu moves, hence the name of the algorithm.

...

4.1.1 Storing a Tabu move

A tabu move is intended as the worsening move that has been done in a previous round, and it basically consists on the 4 edges, or the 2 nodes, that were considered in the swap.
To store the tabu move we have more options:

1. Mark as tabu one of the nodes (fix the two edges connected to that node)

2. Mark as tabu both nodes (fix all four edges in the swap)

3. Mark as tabu one edge or more edges

Marking one or both nodes as tabu moves would restrict our area of search, since after a few tabu moves, lots of edges cannot be changed, so we opted to mark as a tabu move the two edges (p_i, p_{i+1}) and (p_j, p_{j+1}).

4.1.2 The tabu list

The tabu list is intended as the list of tabu moves, that 2-opt will need to consult to see wether a move is admitted or not.
Once the tabu list is filled up, the oldest tabu move will be removed to let the new tabu move be saved.
An important factor regarding the tabu list, is its size: a small size means that the algorithm is not free to explore much the search space, while a big size means that the algorithm will worsen too much the solution, risking in not be able to ever find a better solution.
Another way to look at the size of the tabu list is to consider it as its memory: a small tabu list will forget earlier tabu moves, while a big tabu list will have a longer memory.
The size (of memory) of the tabu list will be referred as the tenure of the tabu list.
We built the tabu list as a fixed length array, where we stored each tabu move together with a counter which increases at each new tabu move.
We used the counter to view if a move is still a tabu move or not: by comparing the current counter with the one stored along the move, we can check how many iterations has passed and if more iterations than the tenure has occurred, that moves is no longer a tabu move.
We tried out two ways of checking the tenure:

1. Static approach: a move in the list is no longer a tabu move if

$$\text{counter} - \text{counter(move)} < \text{tenure}$$

   Where counter(move) is the counter stored along the move in the tabu list.

2. Dinamic approach: a move in the list is no longer a tabu move if

$$\text{counter} - \text{counter(move)} < f(\text{tenure}, \text{counter})$$

   Where $f(\text{tenure}, \text{counter}) := \text{A} \cdot \sin(\text{counter} \cdot \text{B}) + \text{tenure}$, and A, B are parameters given by the user (referred as variable_tenure and variability_frequency).

Here are shown two graphs, plotting the iteration counter and the cost of the solution the algorithm finds itself at:
TODO: Plot cost of tabu.
The dinamic approach performs better for a few reasons:

1. Lowers the risk of getting stuck: with the dinamic approach if the algorithm get stuck, in a few iteration it will start to forget some moves and escapes from that situation

2. Allows for a more dinamic exploration of the search space: the dinamic approach allows the algorithm to "forget" something to look for a better solution in the search space, but the remember it later if that lead to nothing.

4.1.3 Pseudocode

Algorithm 3 tabu algorithm for the TSP

Input Starting node ($s \in V$), Set of nodes (V)
Output List of $n \coloneqq |V|$ nodes forming an Hamiltonian cycle, Cost of the cycle

cycle, cost $\leftarrow$ greedy(s, V)

while *Within the time limit* **do**

$(i, j) \leftarrow$ find_tabu_swap(cycle)
add_to_tabu(i, j)
cost $\leftarrow$ cost $- (c_{p_i,p_{i+1}} + c_{p_j,p_{j+1}}) + (c_{p_i,p_j} + c_{p_{i+1},p_{j+1}})$
reverse(cycle, $i+1, j$)

end while

return cycle, cost

Where the find_tabu_swap() method returns the best swap (allowing for bad moves) after checking the tabu list.

4.1.4 Results analysis

TODO: Comparison with 2-opt

TODO: Comparison between dinamic / static approach

4.2 Variable Neighborhood Search (VNS)

For the tabu algorithm to work, a list of moves must be stored as tabu moves: how many moves to store? Is it better a static or dinamic tenure? In the dinamic approach, how much should the tenure vary? Those are all hyperparameters that should be set through lots of try and error generating the risk of overfitting.
Another approach to metaheuristics that doesn't require hyperparameters, is the Variable Neighborhood Search (VNS), which has the same base idea of the tabu search, but

approaches to it differently.
Once we are in a local minimum, if we make a 2-opt swap (as we do with the tabu algorithm), we must save that move as a tabu move, since the next 2-opt swap will revert it.
The VNS approach is to make a swap that requires more than two edges to be swapped (entering the family of k-opt), in our case a 3-opt swap.
Once a 3-opt swap is performed, it's impossible for a 2-opt swap to reverse that change, since it should change 3 edges and it's allowed to change only 2 of them.
In some scenarios one 3-opt swap is enough to escape the local minimum, but in other it's not and more swaps are needed (an hyperparameter).
To prevent having to set an hyperparameter, we used multithreading to perform dirrerent numbers of 3-opt swaps on a local minimum, then use the 2-opt algorithm to lower the cost, and choose the best among the solutions found.
Another approach that can be explored in the future is using multithreading to keep the k best choices among the solutions found and keep exploring them in parallel (with special attention to keep the list of parallel runs under control, avoiding exponential growth).

4.2.1 Results analysis

TODO: Comparison with 2-opt

4.3 Comparison Tabu / VNS

TODO: Comparison with tabu

Chapter 5

Exact methods

TODO:

5.1 CPLEX

TODO:

5.2 Benders loop

TODO:

5.2.1 Patching

TODO:

5.3 Branch & Cut

TODO:

5.3.1 Warm start

TODO:

5.3.2 Fractionary solutions

TODO:

5.3.3 Patching

TODO:

Chapter 6

Matheuristics

TODO:

6.1 Diving

TODO:

6.2 Local Branching

TODO:

Chapter 7

Bibliography